

nextstep

次のステップ: laplacianFoam で行うソース改造なしデータ同化

この文書の目的

ここまでのサンプルでは、Python で作った 1 次元熱伝導モデルに対してカルマンフィルタを適用しました。

Python 版では、状態方程式が明示的に分かっていました。

$$x_{k+1} = Ax_k + Bu_k$$

そのため、カルマンフィルタの予測ステップをそのまま行列計算で実装できました。

一方、次にやりたいのは OpenFOAM の laplacianFoam を使う場合です。OpenFOAM では、温度場の時間発展はソルバ内部で計算されます。したがって、A や B を明示的に取り出すのではなく、**laplacianFoam そのものを予測モデルとして使う**ことになります。

この文書では、次の方針を提案します。

laplacianFoam のソースコードは改造せず、外部 Python スクリプトから OpenFOAM を逐次実行し、観測データを使ってパラメータを同化する。

特に最初の実装として、**外部制御型の逐次 EnKF 型パラメータ同化**を提案します。

なぜソースコードを改造しない方針にするのか

OpenFOAM ソルバを改造してデータ同化項を入れる方法もあります。たとえば laplacianFoam の方程式に nudging 項を追加する方法です。

しかし最初からソルバを改造すると、次の難しさがあります。

- OpenFOAM のビルド環境が必要
- ソルバの保守が必要
- OpenFOAM のバージョン差分に影響される
- 何が OpenFOAM 本体の問題で、何がデータ同化側の問題か切り分けにくい

- 実験的に手法を変えにくい

そこで最初は、標準の `laplacianFoam` をそのまま使い、外部 Python から次のように制御します。

```
OpenFOAM を次の観測時刻まで進める
↓
センサ位置の温度を取り出す
↓
実験値との差を見る
↓
熱入力倍率や境界条件などのパラメータを更新する
↓
更新した条件で次の観測時刻へ進める
```

この方式なら、OpenFOAM 本体は標準機能だけで使えます。

まず提案する手法

提案する最初の手法は、次です。

外部 Python 制御による逐次 EnKF 型パラメータ同化

ポイントは、温度場 T を直接書き換えることを最初の主目的にしないことです。代わりに、温度場を生み出す原因側のパラメータを推定します。

最初に同化するパラメータとしては、熱入力倍率 `qScale` を推奨します。

```
Q_model = qScale * Q_nominal
```

ここで、

- `Q_nominal`: 事前に想定した熱入力
- `qScale`: 実際の発熱量とのずれを表す倍率

です。

たとえば `qScale = 0.90` なら、発熱量は想定より 10% 小さいと考えます。`qScale = 1.10` なら、発熱量は想定より 10% 大きいと考えます。

パラメータスタディとの違い

ここは重要です。

複数の `qScale` を試すだけなら、それはパラメータスタディです。

`qScale = 0.90` を最後まで計算

qScale = 0.95 を最後まで計算
qScale = 1.00 を最後まで計算
qScale = 1.05 を最後まで計算
qScale = 1.10 を最後まで計算
最後に一番合うものを選ぶ

これはデータ同化ではありません。

データ同化では、観測データが入るたびに、推定値を逐次更新します。

t=0～10s:

複数の qScale 候補で OpenFOAM を進める

t=10s:

センサ値を使って qScale の分布を更新する

t=10～20s:

更新後の qScale で OpenFOAM を進める

t=20s:

またセンサ値で更新する

つまり、データ同化は次のサイクルです。

予測 → 観測 → 更新 → 次の予測

パラメータスタディとの違いは、途中で観測を使って推定値と不確かさを更新し、その更新結果を次の予測に反映することです。

データ同化の理論

状態空間モデル

一般的なデータ同化では、状態方程式と観測方程式を考えます。

状態方程式:

$$x_k = \mathcal{M}_{k-1}(x_{k-1}, \theta_{k-1}) + w_k$$

観測方程式:

$$y_k = \mathcal{H}_k(x_k) + v_k$$

ここで、

- x_k : 時刻 k の状態

- `theta_k`: モデルパラメータ
- `M`: 時間発展モデル
- `H`: 観測演算子
- `y_k`: 観測値
- `w_k`: モデル誤差
- `v_k`: 観測誤差

です。

Python 版では、`M` は行列 `A`, `B` でした。

$$x_{k+1} = Ax_k + Bu_k$$

OpenFOAM 版では、`M` は `laplacianFoam` です。

```
x_{k+1} = laplacianFoam(x_k, theta_k)
```

状態 `x` は、OpenFOAM の全セルの温度場です。

```
x = [T_cell11, T_cell12, ..., T_cell1N]
```

ただし全セル温度場を直接カルマンフィルタで扱うと、セル数が大きすぎます。そこで、最初はパラメータ `theta` を主な同化対象にします。

パラメータを状態に含める

パラメータ同化では、状態ベクトルを次のように拡張します。

$$z_k = \begin{bmatrix} x_k \\ \theta_k \end{bmatrix}$$

たとえば、温度場 `T` と熱入力倍率 `qScale` をまとめて、

```
z = [T field, qScale]
```

と考えます。

最初の簡易版では、`qScale` だけを同化対象にしてもよいです。

```
z = [qScale]
```

この場合、OpenFOAM は「`qScale` を与えると、センサ位置の温度を返すモデル」と見なせます。

```
qScale
↓
laplacianFoam
```

↓

sensor temperature

EnKF の考え方

通常のカルマンフィルタでは、誤差共分散行列 P を持ちます。

$$P \in \mathbb{R}^{N \times N}$$

OpenFOAM のセル数 N が数千、数万になると、この行列を明示的に持つのは現実的ではありません。

そこで、EnKF、つまりアンサンブルカルマンフィルタを使います。

EnKF では、複数のサンプルを用意します。

```
ensemble 1: qScale = 0.90
ensemble 2: qScale = 0.95
ensemble 3: qScale = 1.00
ensemble 4: qScale = 1.05
ensemble 5: qScale = 1.10
```

これは単なるパラメータスタディではなく、qScale の不確かさの分布を表すアンサンブルです。

各アンサンブルを観測時刻まで進めると、センサ位置の予測値もばらつきます。

```
ensemble 1 -> sensor prediction y_1
ensemble 2 -> sensor prediction y_2
ensemble 3 -> sensor prediction y_3
...
```

このばらつきから、パラメータと観測の相関を推定します。

EnKF 更新式

アンサンブルの予測状態を z_i^f 、観測予測を y_i^f とします。

アンサンブル平均は、

$$\bar{z}^f = \frac{1}{N_e} \sum_{i=1}^{N_e} z_i^f$$

$$\bar{y}^f = \frac{1}{N_e} \sum_{i=1}^{N_e} y_i^f$$

です。

偏差行列を、

$$Z' = [z_1^f - \bar{z}^f, z_2^f - \bar{z}^f, \dots]$$

$$Y' = [y_1^f - \bar{y}^f, y_2^f - \bar{y}^f, \dots]$$

とします。

このとき、交差共分散と観測空間の共分散は、

$$P_{zy} = \frac{1}{N_e - 1} Z' Y'^T$$

$$P_{yy} = \frac{1}{N_e - 1} Y' Y'^T + R$$

です。

カルマンゲインは、

$$K = P_{zy} P_{yy}^{-1}$$

となります。

更新式は、

$$z_i^a = z_i^f + K \left(y_k^{\text{obs}} + \epsilon_i - y_i^f \right)$$

です。

ここで、

- f: forecast、OpenFOAM で予測した値
- a: analysis、観測で補正した値
- epsilon_i: 観測ノイズに対応する摂動
- R: 観測ノイズ共分散

です。

z に qScale を含めておけば、観測値との差に応じて qScale も更新されます。

laplacianFoam で行う具体的な手順

全体構成

ケース構成は次のようにします。

```

openfoam_da_case/
├── baseCase/
│   ├── 0/T
│   ├── constant/
│   └── system/
├── da/
│   ├── observations.csv
│   ├── sensor_locations.csv
│   ├── run_enkf.py
│   └── ensembles/
│       ├── ens_000/
│       ├── ens_001/
│       ├── ens_002/
│       └── ...

```

baseCase は通常の laplacianFoam ケースです。ここは OpenFOAM 標準の作り方で構いません。

da/run_enkf.py が外部制御スクリプトです。

観測データ

observations.csv は、たとえば次の形式にします。

```

time,sensor0,sensor1
10,25.1,23.8
20,26.0,24.4
30,26.7,25.0

```

sensor_locations.csv は、センサ位置を記録します。

```

name,x,y,z
sensor0,0.0,0.0,0.0
sensor1,0.1,0.0,0.0

```

Step 1: アンサンブルを作る

baseCase を複製して、複数の OpenFOAM ケースを作ります。

```

ens_000: qScale = 0.90
ens_001: qScale = 0.95
ens_002: qScale = 1.00
ens_003: qScale = 1.05
ens_004: qScale = 1.10

```

このとき、各ケースの qScale をどこに書くかを決めます。

候補は次です。

- constant 配下の自作 dictionary
- system/fvOptions の発熱量
- 境界条件ファイル
- Python 側でテンプレートから 0/T や constant を書き換える

laplacianFoam 標準だけで内部発熱を扱いにくい場合は、最初は境界温度や境界熱流束に相当する条件を qScale で変えるのが扱いやすいです。

Step 2: 各アンサンブルを観測時刻まで実行する

各ケースの system/controlDict を、次の観測時刻まで進むように書き換えます。

```
startTime = 前回の観測時刻
endTime   = 今回の観測時刻
```

そして、各ケースで laplacianFoam を実行します。

```
laplacianFoam -case da/ensembles/ens_000
laplacianFoam -case da/ensembles/ens_001
...
```

Python からは subprocess.run() で呼び出します。

Step 3: センサ位置の温度を取り出す

OpenFOAM 標準の postProcess や sample を使って、センサ位置の温度を取り出します。

例:

```
postProcess -case da/ensembles/ens_000 -func sample -time 10
```

または、sampleDict を用意しておきます。

```
sets
(
    sensors
    {
        type    cloud;
        axis    xyz;
        points
        (
            (0.0 0.0 0.0)
            (0.1 0.0 0.0)
        );
    }
);
```



```
fields (T);
```

各アンサンブルについて、センサ予測値 y_i^f を得ます。

Step 4: EnKF で qScale を更新する

最初は $z_i = qScale_i$ として実装します。

アンサンブルの qScale とセンサ予測 y_i から、

```
Z = q_scales.reshape(n_ens, 1)
Y = sensor_predictions
```

を作ります。

Y は形状 (n_ens, n_sensors) です。

更新の擬似コードは次です。

```
def enkf_update_parameters(q_scales, y_pred, y_obs, r_std):
    # q_scales: shape (n_ens,)
    # y_pred   : shape (n_ens, n_sensors)
    # y_obs    : shape (n_sensors,)

    n_ens = len(q_scales)
    Z = q_scales[:, None]
    Y = y_pred

    z_mean = Z.mean(axis=0)
    y_mean = Y.mean(axis=0)

    Zp = Z - z_mean
    Yp = Y - y_mean

    Pzy = Zp.T @ Yp / (n_ens - 1)
    Pyy = Yp.T @ Yp / (n_ens - 1)
    R = np.eye(Y.shape[1]) * r_std**2

    K = Pzy @ np.linalg.inv(Pyy + R)

    updated = []
    for i in range(n_ens):
        obs_perturb = np.random.randn(Y.shape[1]) * r_std
        innovation = (y_obs + obs_perturb) - Y[i]
        z_new = Z[i] + K @ innovation
        updated.append(float(z_new[0]))

    return np.array(updated)
```

この更新により、観測値に合う方向へ qScale のアンサンブルが移動します。

Step 5: 更新後の qScale を次の計算に反映する

更新後の qScale を、各アンサンブルケースの入力ファイルに書き込みます。

```
ens_000: qScale = updated_qScale_000
ens_001: qScale = updated_qScale_001
...
```

そして次の観測時刻へ進みます。

```
t = 10s で更新
↓
t = 20s まで OpenFOAM
↓
t = 20s で更新
```

これを時系列で繰り返します。

具体的な Python プログラム案

最初の run_enkf.py は、次のような構造にします。

```
from pathlib import Path
import shutil
import subprocess
import numpy as np
import pandas as pd

BASE_CASE = Path("../baseCase")
ENSEMBLE_DIR = Path("ensembles")
N_ENS = 10
OBS_STD = 0.2

def make_ensembles(q_scales):
    ENSEMBLE_DIR.mkdir(exist_ok=True)
    for i, q in enumerate(q_scales):
        case_dir = ENSEMBLE_DIR / f"ens_{i:03d}"
        if case_dir.exists():
            shutil.rmtree(case_dir)
        shutil.copytree(BASE_CASE, case_dir)
        write_qscale(case_dir, q)
```

```

def write_qscale(case_dir: Path, q_scale: float):
    # 実装方針に応じて、fvOptions や境界条件を書き換える。
    # まずは専用 dictionary やテンプレートファイルを使うと安全。
    path = case_dir / "constant" / "qScale"
    path.write_text(f"{q_scale:.8f}\n")

def set_time_window(case_dir: Path, start: float, end: float):
    control_dict = case_dir / "system" / "controlDict"
    text = control_dict.read_text()
    text = replace_dict_entry(text, "startTime", start)
    text = replace_dict_entry(text, "endTime", end)
    control_dict.write_text(text)

def replace_dict_entry(text: str, key: str, value: float) -> str:
    lines = []
    for line in text.splitlines():
        stripped = line.strip()
        if stripped.startswith(key):
            lines.append(f"{key} {value};")
        else:
            lines.append(line)
    return "\n".join(lines) + "\n"

def run_openfoam(case_dir: Path):
    subprocess.run(
        ["laplacianFoam", "-case", str(case_dir)],
        check=True,
    )

def sample_sensors(case_dir: Path, time_value: float) -> np.ndarray:
    subprocess.run(
        ["postProcess", "-case", str(case_dir), "-func", "sample", "-time", str(time_value)],
        check=True,
    )
    # sample 出力の読み取り処理を書く。
    # OpenFOAM の出力形式に合わせて実装する。
    return read_sample_output(case_dir, time_value)

def read_sample_output(case_dir: Path, time_value: float) -> np.ndarray:
    # 例: postProcessing/sample/<time>/sensors_T.xy などを読む。
    raise NotImplementedError

def enkf_update_parameters(q_scales, y_pred, y_obs, r_std):
    n_ens = len(q_scales)

```

```

Z = q_scales[:, None]
Y = y_pred

z_mean = Z.mean(axis=0)
y_mean = Y.mean(axis=0)

Zp = Z - z_mean
Yp = Y - y_mean

Pzy = Zp.T @ Yp / (n_ens - 1)
Pyy = Yp.T @ Yp / (n_ens - 1)
R = np.eye(Y.shape[1]) * r_std**2
K = Pzy @ np.linalg.inv(Pyy + R)

updated = []
for i in range(n_ens):
    obs_perturb = np.random.randn(Y.shape[1]) * r_std
    innovation = (y_obs + obs_perturb) - Y[i]
    z_new = Z[i] + K @ innovation
    updated.append(float(z_new[0]))

return np.array(updated)

def main():
    obs = pd.read_csv("observations.csv")

    q_scales = np.random.normal(loc=1.0, scale=0.1, size=N_ENS)
    make_ensembles(q_scales)

    current_time = 0.0

    for _, row in obs.iterrows():
        next_time = float(row["time"])
        y_obs = row.drop(labels=["time"]).to_numpy(dtype=float)

        y_pred_all = []

        for i in range(N_ENS):
            case_dir = ENSEMBLE_DIR / f"ens_{i:03d}"
            set_time_window(case_dir, current_time, next_time)
            write_qscale(case_dir, q_scales[i])
            run_openfoam(case_dir)
            y_pred = sample_sensors(case_dir, next_time)
            y_pred_all.append(y_pred)

        y_pred_all = np.asarray(y_pred_all)
        q_scales = enkf_update_parameters(q_scales, y_pred_all, y_obs, OBS_STD)

    for i in range(N_ENS):

```

```

write_qscale(ENSEMBLE_DIR / f"ens_{i:03d}", q_scales[i])

print(f"time={next_time}, qScale mean={q_scales.mean():.4f}, std={q_scales.std():.4f}")
current_time = next_time

if __name__ == "__main__":
    main()

```

このコードは設計案です。実際には、OpenFOAM ケースで熱入力をどう与えるか、`sample` の出力形式がどうなるかに合わせて調整が必要です。

外部 Python 制御による逐次 EnKF 型パラメータ同化の何がうれしいか

1. OpenFOAM 標準ソルバをそのまま使える

laplacianFoam のソースコードを改造しないため、OpenFOAM 標準のワークフローを保てます。ソルバ改造の負担がなく、ケース設定と外部 Python だけで実験できます。

2. モデル誤差をパラメータとして扱える

現実の熱解析では、発熱量や境界条件が正確に分からないことが多いです。この手法では、それらを未知パラメータとして扱い、観測データから逐次補正できます。

これは、単に温度場を合わせるだけでなく、

なぜモデルがずれたのか

どのパラメータが現実と合っていないのか

を考える手がかりになります。

3. 温度場の物理的整合性を保ちやすい

温度場を直接書き換えると、不自然な勾配や境界条件との矛盾が出る可能性があります。

パラメータを更新してから OpenFOAM で再計算すれば、温度場は熱伝導方程式に従って生成されます。これは研究として説明しやすい利点です。

4. 実験データと解析モデルをつなげられる

実験センサ値だけでは、測っていない場所の温度は分かりません。OpenFOAM だけでは、モデル条件がずれると現実から離れます。

データ同化は、この 2 つをつなぎます。

OpenFOAM:

空間全体の物理的な温度分布を予測する

実験センサ:

現実とのずれを教えてくれる

データ同化:

モデルを現実へ引き戻す

5. センサ配置の研究へ発展できる

2点センサでどこまで推定できるか、3点ならどこに置くべきか、温度センサと変位センサをどう組み合わせるべきか、といった研究に発展できます。

これは工作機械の熱変位補償や、構造物の状態推定に直結します。

6. デジタルツインの入口になる

OpenFOAM モデルを実験データで逐次補正できるようになると、実機の状態に追従する熱デジタルツインに近づきます。

これは単なる事後解析ではなく、

実験中・運転中にモデルを更新し続ける

という方向です。

研究としての意義

この手法の研究上の意義は、次のように整理できます。

限られたセンサから全体場を推定する

全セルにセンサを置くことはできません。少数センサから全体の温度場や熱変位を推定できれば、計測コストを抑えながら状態把握ができます。

物理モデルと実測データを統合する

純粋なシミュレーションは、境界条件や発熱量の不確かさに弱いです。純粋な計測は、センサのない場所を知ることができません。

データ同化は、両者の弱点を補います。

モデルパラメータの不確かさを扱える

熱入力倍率、境界条件、材料物性などを同化対象にすることで、単なる温度補正ではなく、モデルの不確かさを推定できます。

これは、より信頼性の高い熱解析や補償モデル構築につながります。

ソース改造なしで再現性が高い

OpenFOAM 標準ソルバと外部 Python で構成できるため、他の環境でも再現しやすく、手法比較もしやすいです。

最初の実装するミニマム計画

最初から大きな EnKF を作るのではなく、次の順番で進めるのがよいです。

Phase 1: laplacianFoam 単体ケースを作る

- 1 次元または簡単な 3 次元メッシュを作る
- laplacianFoam で温度拡散問題を解く
- センサ位置に相当する点を決める
- sample で温度を取り出せるようにする

Phase 2: 観測データを用意する

最初は実験データでなくてもよいです。

- 別条件の OpenFOAM 結果を「仮想真値」とする
- そこからセンサ位置だけを抜き出す
- ノイズを足して observations.csv を作る

Python 版でやった True と同じ考え方です。

Phase 3: qScale の逐次推定を実装する

- qScale のアンサンブルを作る
- 各ケースを観測時刻まで進める
- センサ温度を読み取る
- EnKF 更新で qScale を更新する
- 更新後の qScale で次時刻へ進める

Phase 4: 実験データへ適用する

仮想データで動作確認できたら、実験センサ値に置き換えます。

このとき、評価用に一部のセンサを同化に使わず、検証用として残すとよいです。

Phase 5: 同化対象を広げる

次のように発展できます。

- qScale だけでなく境界温度も同化する
- 熱拡散率や熱伝導率も同化する
- 温度場 T も EnKF で補正する
- 変位センサを観測に追加する
- センサ配置最適化へ進む

まとめ

laplacianFoam を使ってソース改造なしでデータ同化を行うなら、最初に提案する手法は次です。

外部 Python 制御による逐次 EnKF 型パラメータ同化

最初の同化対象は、熱入力倍率 qScale がよいです。

この手法は、単なるパラメータスタディではありません。観測時刻ごとにアンサンブルを更新し、その更新結果を次の OpenFOAM 計算へ反映することで、時系列データ同化になります。

この方法のうれしさは、次の通りです。

- OpenFOAM の標準ソルバを改造せずに使える
- 実験データでモデル誤差を逐次補正できる
- 温度場を直接いじるより物理的整合性を保ちやすい
- 少数センサから全体温度場や熱変位の推定へ発展できる
- 工作機械の熱変位補償や熱デジタルツインにつながる

次に実装するなら、まずは小さな laplacianFoam ケースで、qScale の逐次 EnKF 同化を作るのがよいです。

温度場を直接補正する場合との違い

温度場を直接補正する方法では、次のようにします。

$$T^{\text{new}}(x) = T^{\text{foam}}(x) + w(x) (T^{\text{obs}} - T^{\text{foam}}(x_s))$$

これは分かりやすいですが、厳密にはカルマンフィルタではありません。距離重みを人間が決めるため、nudging や経験的補正に近い方法です。

一方、パラメータ同化では、

温度場を直接いじらない

↓

熱入力倍率や境界条件を更新する

↓

OpenFOAM が物理方程式に従って温度場を作る

という流れになります。

この方が、物理的な整合性を保ちやすいです。

参考: ソルバを改造する場合はどうするか

ここまでは「ソースコードを改造しない」方針を主に説明しました。一方で、研究としてより OpenFOAM 内部に近い形でデータ同化を入れるなら、laplacianFoam をコピーして独自ソルバを作り、温度方程式に観測へ引き寄せる項を追加する方法があります。

代表的なのは nudging です。

nudging とは何か

nudging は、計算値が観測値からずれたときに、方程式の中で観測値へ少しずつ引き戻す方法です。

laplacianFoam が解く基本式を、単純化して次のように書きます。

$$\frac{\partial T}{\partial t} = \nabla \cdot (\alpha \nabla T)$$

ここに観測へ引き寄せる項を入れます。

$$\frac{\partial T}{\partial t} = \nabla \cdot (\alpha \nabla T) + \gamma(\mathbf{x}) (T_{\text{obs}} - T)$$

ここで、

- T: OpenFOAM が計算している温度
- T_obs: センサ観測値
- gamma(x): 観測へどれくらい強く引き寄せるかを表す係数

です。

センサが複数ある場合は、

$$\frac{\partial T}{\partial t} = \nabla \cdot (\alpha \nabla T) + \sum_{s=1}^{N_s} \gamma_s(\mathbf{x}) (T_{\text{obs},s} - T)$$

のようにできます。

ただし、このままでは各セルで T_{obs} をどう定義するかが曖昧です。そこで普通は、センサ位置からの距離に応じて観測の影響を広げます。

$$\gamma_s(\mathbf{x}) = \gamma_0 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_s\|^2}{2L^2}\right)$$

ここで、

- $\mathbf{x}_s$: センサ位置
- L : 補正の空間スケール
- γ_0 : nudging の強さ

です。

この方法では、センサに近いセルは強く補正され、遠いセルは弱く補正されます。

この方法はカルマンフィルタなのか

nudging は、厳密にはカルマンフィルタではありません。

カルマンフィルタでは、補正量は誤差共分散から決まります。

$$x^a = x^f + K(y - Hx^f)$$

$$K = PH^T(HPH^T + R)^{-1}$$

一方、nudging では、補正の強さ γ や空間スケール L を人間が設定します。

したがって、nudging は、

観測値へ緩やかに引き寄せるデータ同化手法

ではありますが、

カルマンフィルタ系のデータ同化

とは区別した方がよいです。

ただし、OpenFOAM ソルバに組み込む方法としては、nudging は分かりやすく、実装もしやすいです。

改造方針

laplacianFoam を直接編集するのではなく、コピーして別名のソルバを作るのが安全です。

```
laplacianFoam
↓ コピー
laplacianFoamDA
```

作業の流れは次です。

1. laplacianFoam のソースをユーザー領域へコピーする
2. ソルバ名を laplacianFoamDA に変更する
3. 温度方程式 TEqn に nudging ソース項を追加する
4. 観測値やセンサ位置を読む dictionary を追加する
5. wmake でビルドする
6. 通常の OpenFOAM ケースで laplacianFoamDA を実行する

ソルバコピーの例

OpenFOAM の環境によりパスは異なりますが、概念的には次のようにします。

```
mkdir -p $WM_PROJECT_USER_DIR/applications/solvers/heatTransfer
cp -r $FOAM_APP/solvers/basic/laplacianFoam \
  $WM_PROJECT_USER_DIR/applications/solvers/heatTransfer/laplacianFoamDA
cd $WM_PROJECT_USER_DIR/applications/solvers/heatTransfer/laplacianFoamDA
```

Make/files の実行ファイル名を変更します。

```
laplacianFoam.C
```

```
EXE = $(FOAM_USER_APPBIN)/laplacianFoamDA
```

必要なら、メインファイル名も laplacianFoamDA.C に変えます。

元の TEqn のイメージ

laplacianFoam の温度方程式は、概念的には次のような形です。

```
fvScalarMatrix TEqn
(
    fvm::ddt(T)
    - fvm::laplacian(DT, T)
);

TEqn.solve();
```

ここに nudging 項を追加します。

nudging 項の入れ方

連続式で、

$$\frac{\partial T}{\partial t} = \nabla \cdot (\alpha \nabla T) + S_{\text{nudge}}$$

としたい場合、OpenFOAM の離散式では右辺にソース項を入れます。

```
fvScalarMatrix TEqn
(
    fvm::ddt(T)
  - fvm::laplacian(DT, T)
  ==
    nudgeSource
);
```

ここで nudgeSource は各セルに定義された scalar field です。

```
volScalarField nudgeSource
(
    IOobject
    (
        "nudgeSource",
        runTime.timeName(),
        mesh,
        IOobject::NO_READ,
        IOobject::NO_WRITE
    ),
    mesh,
    dimensionedScalar("zero", T.dimensions()/dimTime, 0.0)
);
```

温度の次元が [K] なら、nudgeSource の次元は [K/s] になります。

センサ 1 点の場合のソース項

1 つのセンサだけを考えると、各セルのソース項は次のようにできます。

```
forAll(mesh.C(), cellI)
{
    const vector x = mesh.C()[cellI];
    const scalar r2 = magSqr(x - sensorPosition);
    const scalar weight = Foam::exp(-r2/(2.0*sqr(lengthScale)));

    nudgeSource[cellI] =
```

```

        gamma0 * weight * (sensorTemperature - T[cellI]);
    }

```

このとき、

- `sensorPosition`: センサ位置
- `sensorTemperature`: センサ観測温度
- `lengthScale`: 観測影響の広がり
- `gamma0`: 観測へ引き寄せる強さ

です。

`gamma0` の単位は $[1/s]$ です。大きすぎると観測へ強く引っ張りすぎて不安定になります。小さすぎるとほとんど効きません。

複数センサの場合

複数センサの場合は、各センサからの補正を足し合わせます。

```

nudgeSource = dimensionedScalar("zero", T.dimensions()/dimTime, 0.0);

forAll(sensorPositions, s)
{
    const vector xs = sensorPositions[s];
    const scalar Ts = sensorTemperatures[s];

    forAll(mesh.C(), cellI)
    {
        const vector x = mesh.C()[cellI];
        const scalar r2 = magSqr(x - xs);
        const scalar weight = Foam::exp(-r2/(2.0*sqr(lengthScale)));

        nudgeSource[cellI] += gamma0 * weight * (Ts - T[cellI]);
    }
}

```

この方法では、全セルの温度がセンサ値の方向へ補正されます。ただし、補正量はセンサからの距離で変わります。

観測値をどう読むか

研究用の最初の実装では、観測値を `dictionary` から読むのが簡単です。

例えば `constant/DAProperties` を作ります。

```

DAProperties
{

```

```

active          true;
gamma0          0.01;    // [1/s]
lengthScale     0.02;    // [m]

sensors
(
    sensor0
    {
        position    (0.0 0.0 0.0);
        temperature 25.0;
    }

    sensor1
    {
        position    (0.1 0.0 0.0);
        temperature 23.5;
    }
);
}

```

ソルバ側では、

```

IOdictionary DADict
(
    IOobject
    (
        "DAProperties",
        runTime.constant(),
        mesh,
        IOobject::MUST_READ_IF_MODIFIED,
        IOobject::NO_WRITE
    )
);

```

として読みます。

MUST_READ_IF_MODIFIED を使えば、時刻ごとに外部から DAProperties を更新し、ソルバが再読み込みする構成も可能です。

時系列観測を扱う場合

実験データは普通、時系列です。

```

time,sensor0,sensor1
10,25.1,23.8
20,26.0,24.4
30,26.7,25.0

```

ソルバ改造でこれを扱う方法は 2 つあります。

■方法 A: 外部 Python が DAProperties を書き換える OpenFOAM ソルバは DAProperties を読むだけにします。

Python:

現在時刻に対応する観測値を DAProperties へ書く

laplacianFoamDA:

DAProperties を読み、nudging 項を計算する

これは実装が簡単です。

■方法 B: ソルバが CSV を読む ソルバ内で CSV を読み、現在時刻に対応する観測値を補間します。

これは外部制御が少なくて済みますが、C++ 側の実装が少し増えます。最初は方法 A をおすすめします。

TEqn への組み込みイメージ

最終的な温度方程式のイメージは次のようになります。

```
while (runTime.loop())
{
    Info<< "Time = " << runTime.timeName() << nl << endl;

    #include "readDAProperties.H"
    #include "calcNudgeSource.H"

    fvScalarMatrix TEqn
    (
        fvm::ddt(T)
        - fvm::laplacian(DT, T)
        ==
        nudgeSource
    );

    TEqn.solve();

    runTime.write();
}
```

実装を整理するなら、メインの .C ファイルに全部書くのではなく、

```
readDAProperties.H
calcNudgeSource.H
```

のように分けると読みやすくなります。

fvOptions で入れる方法もある

OpenFOAM には `fvOptions` というソース項追加の仕組みがあります。条件によっては、ソルバを改造せずにソース項を入れられます。

ただし、観測値との差、

$$T_{\text{obs}} - T$$

のように時々刻々変わる項を、複数センサと距離重みで入れるには、標準の `fvOptions` だけでは表現しづらい場合があります。

そのため、

- 単純な一様ソースを入れるだけなら `fvOptions`
- センサ位置・観測値・距離重みを使うならソルバ改造

という整理がよいです。

ビルド

ソルバを編集したら、次でビルドします。

```
wmake
```

成功すると、ユーザーアプリケーションとして `laplacianFoamDA` ができます。

実行は通常の OpenFOAM ケースで、

```
laplacianFoamDA -case yourCase
```

です。

改造版 nudging の利点

`nudging` をソルバに入れると、次の利点があります。

- OpenFOAM の時間発展と観測補正が同じ方程式の中で扱える
- 時間刻みごとに滑らかに観測へ引き寄せられる
- 温度場を書き換える外部処理より、数値的に自然に見える
- センサ位置、補正半径、補正強さの影響を研究しやすい

改造版 nudging の注意点

一方で、注意点もあります。

- 厳密なカルマンフィルタではない
- `gamma0` と `lengthScale` の選び方に結果が依存する
- 強く補正しすぎると物理方程式より観測への追従が支配的になる
- センサノイズが大きい場合、ノイズまで温度場に入れてしまう
- 境界条件との整合性に注意が必要

特に `gamma0` は重要です。

`gamma0` が大きい:

観測へ速く近づくが、不安定・ノイズ混入のリスクがある

`gamma0` が小さい:

安定だが、補正効果が弱い

改造版と外部 EnKF の位置づけ

両者は目的が少し違います。

方法	特徴	研究上の位置づけ
外部 EnKF 型パラメータ同化	ソース改造なし。パラメータの不確かさを扱いやすい	カルマンフィルタ系データ同化
nudging 改造ソルバ	方程式内に観測緩和項を入れる。実装が直感的	観測緩和型データ同化
温度場直接補正	外部から T ファイルを書き換える	簡易補正・プロトタイプ

研究として「カルマンフィルタを用いた」と言いたいなら、外部 EnKF の方が説明しやすいです。

研究として「OpenFOAM ソルバに観測同化項を組み込んだ」と言いたいなら、nudging 改造ソルバが分かりやすいです。

改造する場合の最初のミニマム実装

最初は、次の範囲に絞るのがよいです。

1. センサ 1 点だけ
2. 観測値は `constant/DAProperties` に固定値で書く
3. Gaussian 重みで全セルに nudging 項を入れる
4. `gamma0` と `lengthScale` を変えて挙動を見る

5. 次にセンサ 2 点へ増やす
6. 最後に時系列観測へ対応する

この順番なら、ソルバ改造の影響を一つずつ確認できます。

改造版の研究テーマ例

改造版 nudging を使うなら、次のような研究テーマにできます。

- nudging 係数 γ_0 が推定精度に与える影響
- 補正半径 lengthScale とセンサ配置の関係
- センサノイズがある場合の安定性
- 温度センサ数を増やしたときの改善量
- nudging と EnKF の比較
- 温度場同化が熱変位推定に与える影響

特に、外部 EnKF 型パラメータ同化と nudging 改造ソルバを比較すると、

パラメータを直すのがよいのか

温度場を観測へ直接引き寄せるのがよいのか

という、研究として面白い比較ができます。